

Multimedia Fusion Flash extension SDK

Welcome to the MMF Flash extension SDK. Please read this file up to the end. After reading it, you will be able to create new extensions for the MMF Flash runtime. If you are fluent with the Java extension kit, you will see that the Flash extension kit is very similar.

This extension kit contains all the source code of the Flash runtime. I (François) would be really happy to hear any suggestion you have about my source code, as it might help me improve it. If you have an idea about the source code, please write it to me :

francois@clickteam.com

Of course, the source code of the Flash runtime is NOT public, or in any kind of GNU licence. It remains the property of Clickteam, and you should not copy it or distribute it in any way.

Getting ready to program.

Flex builder.

The first thing you need to do to create your Flash extension, is to install a copy of Flex Builder on your machine.

Flex Builder is an integrated development environment, aimed at creating Flash or Flex applications. It is based on Eclipse, which ensures a stable and consistent environment. You can download a version of Flex Builder at the following address:

<http://www.adobe.com/cfusion/entitlement/index.cfm?e=flexbuilder3>

You can use Flex builder free for a period of two months. After that time you will have to purchase it, the price being really reasonable.

You may have to install a debug version of Flash player 9 if the installer could not do it. I suggest you try first to run your first Flash application from the IDE. If it works, this means that the player was installed successfully. If not then do the following : open the control panel, and uninstall the current version of your Flash player. Then open the Flex builder folder. In one of the sub-folders, you will find a debug version of the Flash player. Install it. Please note that some anti-virus programs like NOD32 may block the communication between the IDE and the Flash plugin. If you get an error message when running your application, remove IE from the list of blocked applications in the anti-virus program or firewall.

Creating the RuntimeFlash project

Flex builder (as all Eclipse based IDE) does not allow opening an existing project. So you will have to create a new project and import the Flash runtime sources into it.

- Choose the **File / New / MXML Application** option from the menu. Enter "**RuntimeFlash**" as name. This will create a new RuntimeFlash project.
- Open a file window on the project, and locate the "**src**" folder. Then locate the directory where you installed the Flash extension SDK and its "src" folder. Now copy all the files from the Flash SDK src folder into the new RuntimeFlash project src folder.
- Now go back to Flex Builder. Right click on the project name in the project window, and choose "**Refresh**". This will import all the source files in the project.

Creating a new extension.

To create a new extension :

- Right click on the "**Extensions**" folder in the project window.
- Choose "**New / Actionscript class**"
- Enter for the name of your extension, the name "**CRun**" (respecting the case) followed by the file name of your extension. For example if the extension name is kchisc, then the name of the class should be "CRunkchisc"
- Start programming your extension.

You will have to incorporate your extension in the main Flash runtime. So do so :

- Open the file "Extensions / CExtLoad.as"
- In the loadRunObject method, go to the end of the big "if" structure.
- Enter the following code :

```
if (CServices.compareStringsIgnoreCase(name, "Name_Of_Extension")
{
    object=new CRunName_Of_Extension();
}
```

This line will incorporate your extension in the runtime. The name in quotes must be the name of the extension as it is on disc.

Note : your extension file will NOT be compiled if you do not refer to it in the CExtLoad runtime. Any modification will not be taken into account.

You are free to create any classes you want for your extension. But the main thing is that all the class names **MUST** begin with "**CRun**". If you want to create say, a parser class, then call it "CRunParser". This is important if you want your code to be officially incorporated in the main source, the one that I have on my computer.

Please note that the sources of MMF Runtime included in the SDK contain the sources of all the extension made at that time. If you need to see example extensions, just have a look at all the files in the extensions folder.

Running applications with your extension.

To be able to debug your extensions in Flash builder, you need a version of MMF after build 249 beta 20. If you installed the Flash SDK, you will have access to a new feature in MMF : the Flash debug build.

To build such an application, in MMF, hold down the **SHIFT** key while building your application as a Flash application. You must press the SHIFT key when you click Enter in the build file selector, and keep it down until the end of the build (which should be quicker than a normal Flash build). You must, in the file selector, choose the root of your "src" directory of your RuntimeFlash project. You can enter any name you want for the file, this name will be replaced by "Application.ccf".

When you do this, your application is saved as a CCF file the src folder. MMF also computes all the sounds and fonts, and incorporate them in the RuntimeFlash.mxml file. If you edit this file, you should leave the insertion remarks as they are, otherwise the system will not work.

Once you have built your application, you must choose the "Project / Clean" option in Flex Builder to make it build your new application. Once done you will be able to debug your code.

The file RuntimeFlash.mxml contains the definition of the Flash application. It is set to be a 640x480 application at 50 hz. If you want, you can modify this to your desire.

Publishing your extension

Once you are happy with your extension, you need to publish it. To do so :

- Start **DatMaker** from the Flash SDK Start menu
- Choose the files that need to be included, by clicking on "Add new files" and selecting the files in the file selector. This is a multiple file file selector, you can select all the sources of your extension at once.
- Click on "Save the DAT file", and enter the name of your extension. It should be the exact same name as the C++ extension.
- A new .dat file will be created. Copy this file in the Data\Runtime\Flash folder. After that, your extension should be available to the user in Flash build mode.

The extension class

As stated before, the name of this class should be "CRunNameOfExtension". It should be in the Extension package.

The class is derived from an class, name CRunExtension.

The class contains functions similar to the C++ extensions, for the runtime. Only more structured. Lets see the functions in details.

In MMF Flash runtime, all object's name start with "C".

Default variables

Two variables are defined in the class parent from the object :

- ho : points to the CObject object. Equivalent to the headerObject structure in MMF. Ho is also useful as callback functions are defined to exchange data with the main runtime.
- rh : points to the CRun object. Equivalent to the runHeader structure in MMF.

Constructor

Nothing special to do in the constructor of the extension. But you can have your own code in it.

public function getNumberOfConditions():int

This function should return the number of conditions contained in the object (equivalent to the CND_LAST define in the ext.h file).

public function createRunObject(file:CBinaryFile, cob:CCreateObjectInfo, version:int):Boolean

This function is called when the object is created. As Flash object are just created when needed, there is no EDITDATA structure. Instead, I send to the function a CBinaryFile object, pointing directly to the data of the object (the EDITDATA structure, on disc).

The CBinaryFile object allows you to read the data. It automatically converts PC-like ordering (big or little Indian I cant remember) into Flash ordering. It contains functions to read bytes, shorts, int, colors and strings. Read the documentation about this class at the end of the document.

"Version" contains the version value contained in the extHeader structure of the

EDITDATA.

So all you have to do, is read the data from the CBinaryFile object, and initialise your object accordingly.

public function destroyRunObject(bFast:Boolean):void

Called when the object is destroyed. Due to garbage collection, this routine should not have much to do, as all the data reserved in the object will be freed at the next GC. bFast is true if the object is destroyed at end of frame. It is false if the object is destroyed in the middle of the application.

public function handleRunObject():int

Same as the C++ function. Perform all the tasks needed for your object in this function. As the C function, this function returns value indicating what to do :

- CRunExtension.REFLAG_ONESHOT : handleRunObject will not be called anymore
- CRunExtension.REFLAG_DISPLAY : displayRunObject is called at next refresh.
- Return 0 and the handleRunObject method will be called at the next loop.

public function displayRunObject():void

Called to display the object. If your object needs to be displayed on the screen, you need to add it to one of the display lists.

Your object is a sprite

It is a sprite if the HOFLAG_SPRITE flag is set to ON in the C++ code.

You will need to do an addChild to the rh.rhFrame.layers[ho.hoLayer].planeSprites list.

Your object is a control (on top of everything)

You should add it to : rh.rhApp.planeControls.

Your object is a backdrop object (like the background system box)

rh.rhFrame.layers[ho.hoLayer].planeBack

Once your object is added, you should set its coordinates (minus rh.WindowX and Y), and draw whatever you want in it.

public function pauseRunObject():void

Called when the application goes into pause mode.

public function continueRunObject():void

Called when the application restarts.

public function getRunObjectFont():CFontInfo

Equivalent to the C++ version. This function returns a CFontInfo object, a small object equivalent to the LOGFONT structure in C. See at the end of the document the definition of the CFontInfo object.

public function setRunObjectFont(fi:CFontInfo, rc:CRect):void

Called when the font of the object needs to be changed. The rc parameter is null when no resize is needed.

public function getRunObjectTextColor():int

Returns the current color of the text as an Integer.

public function setRunObjectTextColor(int rgb):void

Sets the current color of the text.

public function getRunObjectCollisionMask(flags:int):CMask

If implemented, this function should return a CMask object that contains the collision mask of the object. Return null in any other case.

public function condition(num:int, cnd:CCndExtension):Boolean

The main entry for the evaluation of the conditions.

- num : number of the condition (equivalent to the CND_ definitions in ext.h)
- cnd : a pointer to a CCndExtension object that contains useful callback functions to get the parameters.

This function should return true or false, depending on the condition.

public function action(num:int, act:CActExtension):void

The main entry for the actions.

- num : number of the action, as defined in ext.h
- act : pointer to a CActExtension object that contains callback functions to get the parameters.

public function expression(num:int):CValue

The main entry for expressions.

- num : number of the expression

To get the expression parameters, you have to call the getExpParam() method defined in the "ho" variable, for each of the parameters. This function returns a CValue which contains the parameter. You then do a getInt(), getDouble() or getString() with the CValue object to grab the actual value.

This method should return a CValue object containing the value to return. The content of the CValue can be a integer, a double or a String. There is no need to set the HOF_STRING flags if your return a string : the CValue object contains the type of the returned value.

Callback functions

The "ho" variable in the extension object gives you access to the CExtension object, which is a derivative of the main CObject class. I have programmed a few callback functions in the CExtension object. Here are these functions :

public function getX():int

Returns the current X co-ordinate of the object (hoX).

public function getY():int

Returns the current Y coordinate of the object (hoY)

public function getWidth():int

Returns the current width of the object (holmgWidth).

public function getHeight():int

Returns the current height of the object (holmgHeight).

public function setPosition(x:int, y:int):void

Changes the position of the object, taking the movement into account (much better than poking into hoX and hoY).

public function setX(x:int):void

Changes the position of the object (hoX), and takes care of the movement and refresh. Same remark as setPosition.

public function setY(y:int):void

Same as setX for Y co-ordinate. Same remark as setPosition.

public function setWidth(width:int):void

Change the width of the object, taking care of the hoRect fields.

public function setHeight(height:int):void

Same as setWidth, for height.

public function scaleX(x:int):int

Calculates the resulting coordinate taking the stretch to fill display property. If the frame is not stretched, this method has no effect, if it has, it calculates the correct coordinate.

public function scaleY(int:int):int

Same as before, for the Y coordinate.

public function loadImageList(imageList:Array):void

This method should be called in the createRunObject method of your object. If your object uses images stored in the image bank, you must call this method so that the proper images are loaded.

Just make an array with all the handles of the images, the size should be the exact number of images to load. Call this method (it may take some time to return). All the images will be loaded in the runtime.

public function getImage(handle:int):CImage

Call this function to retrieve an image from a handle. The image must have been previously loaded with loadImageList. The value returned could be null if the image could not be loaded, but this is very unlikely to occur.

CImage is part of the "Banks" package. So you should import it with "import Banks.*".

public function reHandle():void

If you returned a REFLAG_ONESHOT value in your handleRunObject method, this will reinforce the method to be called at each loop.

public function generateEvent(code:int, param:int):void

Generate an event with the specific code. The parameter can be recuperated with the ho.getEventParam method. You should prefer the push_event method, specially if your event is generated in a listener (MouseListener, EventListener) as the listener events occurs in another thread and the event routines are not multi-thread proof.

public function pushEvent(code:int, param:int):void

This is the method of choice to generate an event.

public function pause():void

Pauses the application, when you have some lengthy work to perform (like opening a file selector).

public function resume():void

Resumes the application at the end of your work.

public function redraw():void

Forces a redraw of the object (the displayRunObject routine is called at next refresh).

public function destroy():void

Destroys the object at the end of the current loop.

public function getExtUserData():int

Returns the private field of the extHeader structure.

public function setExtUserData(data:int):void

Changes the private field of the extHeader structure.

public function getEventCount():int

Returns the rh4EventCount value, used in controls to trigger the events.

public function getExpParam():CValue

Returns the next expression parameter.

public function getFirstObject():CObject

Returns the first object currently defined in the frame. Should be used in conjunction with getNextObject().

public function getNextObject():CObject

Returns the next object in the list of objects of the frame. Returns null if no more objects is available. This method will return the extension object it is called from.

CRun callback functions

You access the CRun object via the "rh" variable defined in the CRunExtension class. This object contains three methods used to define global data.

Some extensions need to communicate between objects. In C++ this was done simply by defining global variable in the code. I have defined three functions in CRun to allow you to create global classes.

public function addStorage(data:CExtStorage, id:int):void

Adds a new object to the storage, with the "id" identifier. "id" is a simple integer number. The storage class must be derived from the class CExtStorage. This function has no effect if an object with the same identifier already exists.

public function getStorage(id:int):CExtStorage

Returns the storage object with the given identifier. Returns null if the object is not found.

public function delStorage(id:int):void

Deletes the object with the given identifier.

CActExtension callback functions

The CActExtension object is transmitted to the extension "action" method when a action is called. This object contains callback function to gather the parameters of the action.

These functions want two parameters:

- The CRun object (available in the extension as "rh").
- The number of the parameter in the action, starting at 0

public function getParamObject(rhPtr:CRun, num:int):CObject

Returns the CObject pointed to by the PARAM_OBJECT.

public function getParamTime(rhPtr:CRun, num:int):int

Returns the time value in milliseconds.

public function getParamBorder(rhPtr:CRun, num:int):int

Returns the border parameter.

public function getParamDirection(rhPtr:CRun, num:int):int

(obsolete, but might be used in old extensions). Returns a direction from 0 to 31.

public function getParamCreate(rhPtr:CRun, num:int):PARAM_CREATE

Returns a pointer to the PARAM_CREATE object (for future use maybe).

public function getParamAnimation(rhPtr:CRun, num:int):int

Returns the number of the animation.

public function getParamPlayer(rhPtr:CRun, num:int):int

Returns the number of the player.

public function getParamEvery(rhPtr:CRun, num:int):PARAM_EVERY

Returns a pointer to the Every parameter.

public function getParamKey(rhPtr:CRun, num:int):int

Return the key code contained in the parameter.

public int getParamSpeed(rhPtr:CRun, num:int)

Returns a speed, from 0 to 100.

public function getParamPosition(rhPtr:CRun, num:int):CPositionInfo

Returns a pointer to a CPositionInfo classe that contains the X and Y coordinate. CPositionInfo is defined in the Params package.

public function getParamJoyDirection(rhPtr:CRun, num:int):int

Returns a joystick direction.

public function getParamShoot(rhPtr:CRun, num:int):PARAM_SHOOT

Returns a pointer to the PARAM_SHOOT object contained in the action (for future use maybe).

public function getParamZone(rhPtr:CRun, num:int):PARAM_ZONE

Returns a pointer to the PARAM_ZONE parameter.

public function getParamExpression(rhPtr:CRun, num:int):int

Returns the value contained in the expression.

public function getParamColour(rhPtr:CRun, num:int):int

Returns a color, as an integer.

public function getParamFrame(rhPtr:CRun, num:int):int

Returns a number of frame.

public function getParamNewDirection(rhPtr:CRun, num:int):int

Returns a direction, from 0 to 31.

public short getParamClick(rhPtr:CRun, num:int)

Returns the click parameter (left/middle/right button).

public function getParamFilename(rhPtr:CRun, num:int):String

Returns a filename as a string.

public function getParamExpString(rhPtr:CRun, num:int):String

Returns the result of a String expression.

public function getParamExpDouble(rhPtr:CRun, num:int):double

Returns as a double the result of the evaluation of the parameter.

public function getParamFilename2(rhPtr:CRun, num:int):String

Returns the filename as a String object.

CCndExtension callback functions

The CCndExtension object is transmitted to the extension when calling the condition method. It contains callbacks to gather the parameters of the condition. Most of the methods are identical to the ones in CActExtension, with the following differences :

Missing methods:

getParamPosition
getParamCreate
getParamShoot

Different returns

The getParamObject method returns a pointer to the PARAM_OBJECT object (as in the C++).

public function getParamObject(rhPtr:CRun, num:int):PARAM_OBJECT

public function compareValues(rhPtr:CRun, num:int, value:CValue):Boolean

In the C++ version, when you had a PARAM_COMPARISON parameter in the condition, the condition routine returned a long value, and MMF was automatically doing the comparison with the parameter. You have to call this method in the Flash version. For example, a condition with a PARAM_COMPARISON as first parameter, the end of the condition method should be:

```
return cnd.compareValues(rh, 0, returnValue);
```

Where returnValue is a CValue object containing the value to compare with.

public function compareTime(rhPtr:CRun, num:int, t:int):Boolean

Same as the previous function, for PARAM_CMPTIME parameters, where "t" is the time to compare to.

Useful objects

I will document now some of the objects of the runtime you will be using to program your extension.

The CBinaryFile object

This object is sent to you in the createRunObject method. It automatically points to the start of the extension data (EDITDATA structure, in memory). It automatically performs the indian translation between PC values and Flash values.

public function readByteArray(length:int):ByteArray

Read a array of bytes.

public function skipBytes(n:int):void

Skips bytes in the file.

public function getFilePointer():int

Returns the current file pointer position.

public function seek(pos:int):void

Change the file pointer position.

public function readByte():int

Reads one byte.

public function readShort():int

Reads one short (two bytes).

public function readInt():int

Reads an integer (4 bytes).

public function readChar():int

Reads an char (2 bytes).

public function readCharArray(size:int):Array

Reads an array of chars.

public function readColor():int

Reads a color making it compatible with Flash color values (inversion of Red and Blue values).

public function readString():String

Reads a string that finishes with 0.

public function readStringSize(int size):String

Reads a string of the given size.

public function readLogFont():CFontInfo

Reads a LOGFONT structure into a CFontInfo object.

public function readLogFont16():CFontInfo

Reads a LOGFONT structure into a CFontInfo object in the old 16 bits format.

public function skipString():void

Skips a string ending with 0.

The CValue object

The CValue object is used in expression evaluation. It can contain an Integer, a double or a String.

public CValue(int i)

Creates the object as type integer with the given value.

public function getType():int

Returns the type of the object. The return value can be :

CValue.TYPE_INT : and integer

CValue.TYPE_DOUBLE : a double

CValue.TYPE_STRING : a string

public function getInt():int

Returns an integer. (if the object is of TYPE_DOUBLE, converts the value to int).

public function getDouble():Number

Returns a double.

public function getString():String

Returns the string. Warning, if the object is of type int or double, this will NOT convert the value to string.

public function forceInt(value:int):void

Forces the value of type int into the object (if the object was previously double or int, then changes the type).

public function forceDouble(value:Number):void

Forces the value of type double.

public function forceString(value:String):void

Forces a string.

public function forceValue(value:CValue):void

Forces the content and type of the CValue.

public function setValue(value:CValue):void

Change the content of the object, respecting its type.

The CFontInfo object

The CFontInfo object is a replacement of the LOGFONT structure in C++. It contains the name of the font, its height, weight and attributes.

Fields contained in the object :

```
public var lfHeight:int=0;
public var lfWeight:int=0;
public var lfItalic:int=0;
public var lfUnderline:int=0;
public var lfStrikeOut:int=0;
public var lfFaceName:String=null;
```

public function getTextFormat():TextFormat

Returns a TextFormat object based on the characteristics of the font.

The CRect object

This object is intended as a replacement of the C++ RECT structure.

Fields :

```
public var left:int=0;
public var top:int=0;
public var right:int=0;
public var bottom:int=0;
```

public function copyRect(srce:CRect):void

Copies the content of the given CRect object in the object.

public function ptInRect(x:int, y:int):Boolean

Returns true if the given point is located in the rectangle.

public function intersectRect(rc:CRect):Boolean

Returns true if the two rectangles intersect.

The CPoint object

The CPoint object is a replacement of the C++ POINT structure. It only contains two fields:

```
public var x:int=0;
public var y:int=0;
```